J Bot Referral program (EVM Smart Contracts)

Security Audit Report

December 5, 2025

Copyright © 2025 BugBlow. All rights reserved.

No part of this publication, in whole or in part, may be reproduced, copied, transferred or any other right reserved to its copyright owner, including photocopying and all other copying, any transfer or transmission using any network or other means of communication, any broadcast for distant learning, in any form or by any means such as any information storage, transmission or retril system, without prior written permission from BugBlow.

1. INTRODUCTION 3
1.1 Disclaimer 3
1.2 Security Assessment Methodology 3
1.2.1 Code and architecture review: 3
1.2.2 Check code against vulnerability checklist: 3
1.2.3 Threat & Attack Simulation: 4
1.2.4 Report found vulnerabilities: 4
1.2.5 Fix bugs & re-audit code: 4
1.2.6. Deliver final report: 4
Severity classification 5
1.3 Project Overview 5
1.4 Project Dashboard 6
Project Summary 6
Project Last Log 6
Project Scope 6
1.5 Summary of findings 7
2. FINDINGS REPORT 8
2.1 Critical 8
2.2 High 8
2.2.1 Arbitrary User-Supplied *executorData* Not Authorized by Operator Signature 8
2.3 Medium 10
2.4 Low 10
CONCLUSION 10
REFERENCES 10

1. INTRODUCTION

1.1 Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only. The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the Client. If you are not the intended recipient(s) of this document, please note that any disclosure, copying or dissemination of its content is strictly forbidden.

1.2 Security Assessment Methodology

BugBlow utilized a widely adopted approach to performing a security audit. Below is a breakout of how our team was able to identify and exploit vulnerabilities.

1.2.1 Code and architecture review:

- Review the source code.
- Read the project's documentation.
- Review the project’s architecture.

Stage goals

- Build a good understanding of how the application works.

1.2.2 Check code against vulnerability checklist:

- Understand the project’s critical assets, resources, and security requirements.
- Manual check for vulnerabilities based on the auditor's checklist.
- Run static analyzers.

Stage goals

- Identify and eliminate typical vulnerabilities (gas limit, replay, flash-loans attack, etc.)

1.2.3 Threat & Attack Simulation:

- Analyze the project's critical assets, resources and security requirements.
- Exploit the found weaknesses in a safe local environment.
- Document the performed work.

Stage goals

- Identify vulnerabilities that are not listed in static analyzers that would likely be exploited by hackers.
- Develop Proof of Concept (PoC).

1.2.4 Report found vulnerabilities:

- Discuss the found issues with developers
- Show remediations.
- Write an initial audit report.

Stage goals

- Verify that all found issues are relevant.

1.2.5 Fix bugs & re-audit code:

- Help developers fix bugs.
- Re-audit the updated code again.

Stage goals

- Double-check that the found vulnerabilities or bugs were fixed and were fixed correctly.

1.2.6. Deliver final report:

- The Client deploys the re-audited version of the code
- The final report is issued for the Client.

Stage goals

- Provide the Client with the final report that serves as security proof.

Severity classification

All vulnerabilities discovered during the audit are classified based on their potential severity and have the following classification:

Severity	Description
Critical	Vulnerabilities leading to assets theft, fund access locking, or any other significant loss of funds.
High	Vulnerabilities that can trigger contract failure or small loss of funds. Further recovery is possible only by modification of the contract state or replacement.
Medium	Vulnerabilities that can break the intended contract logic or expose it to DoS attacks, but do not cause direct loss funds.
Low	Vulnerabilities that do not have a significant immediate impact and could be easily fixed.

1.3 Project Overview

This project implements a smart-contract based referral fee system integrated with a swap execution layer. The protocol consists of two main components: an Exchange contract, responsible for executing operator-authorized swap orders and collecting protocol and referral fees, and a ReferralFeeVault, which tracks referral rewards and enables their claiming. Swap execution is delegated to trusted executor contracts that integrate with external DEXes. Referral rewards are accumulated per logical user identifier and can only be claimed through operator-signed authorization.

1.4 Project Dashboard

Project Summary

Title	Security audit
Client name	JBot
Project name	Referral Program SC (EVM)
Timeline	17.11.2025 - 04.12.2025
Number of Auditors	3

Project Last Log

Date	Commit Hash	Commit Note
Nov 12, 2025	6a7c92d60a9319925e8848e1601aa8eab5ef632e	deploy: account to BNB Smart Chain Mainnet

Project Scope

The audit covered the following files.

Files
src/exchange/OrderValidator.sol
src/exchange/ Exchange.sol
src/executors/OKXExecutors.sol
src/libraries/CallExecutor.sol

src/utils/Constants.sol
src/utils/TokenHelpers.sol
src/vaults/libraries/ClaimValidator.sol
src/vaults/libraries/ProtocolFeeVault.sol
src/vaults/ReferralFeeVault.sol

1.5 Summary of findings

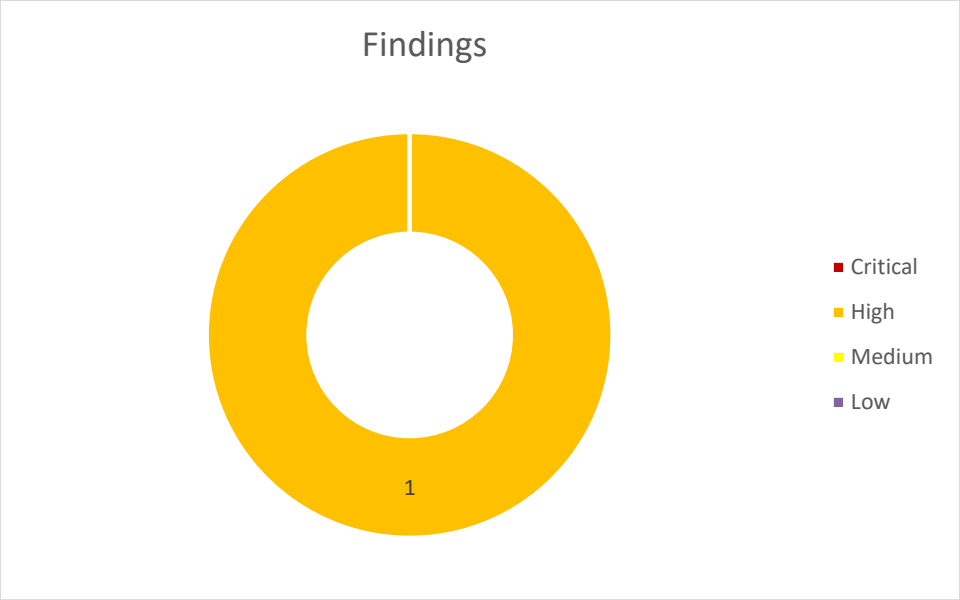

Figure 1. Findings chart

Severity	# of Findings
CRITICAL	0
HIGH	1
MEDIUM	0
LOW	0

2. FINDINGS REPORT

2.1 Critical

Not found

2.2 High

2.2.1 Arbitrary User-Supplied *executorData* Not Authorized by Operator Signature

Status: Fixed

Description

In the Exchange contract, the operator (backend) signed only the economic parameters of an order, namely:

- $tokenIn, tokenOut, amountIn, minAmountOut, referralFees$

However, the execution method of the order — specifically:

- executor
- executorData (router address, calldata, approval targets, swap paths, etc.)

were **not included in the operator signature** and were not validated on-chain.

Order execution was performed via:

$$CallExecutor.delegateCall(executor, executorData);$$

Because this uses delegatecall, the executor code is executed in the context of the Exchange contract, inheriting:

- its storage,
- its token balances,
- and its ERC20 allowances.

As a result, the operator signature authorizes *what* trade should occur economically,
but does not authorize *how* the trade is executed on-chain.

Why this is a problem:

1. A malicious actor can submit arbitrary *executorData* while using a valid operator-signed order;

2. They can use unexpected routers or swap paths;

3. They can extract residual or accidentally sent tokens from the Exchange contract by abusing approvals.

As a result the protocol loses on-chain control over *how* swaps are executed,

Problematic code:

Exchange.sol

```
CallExecutor.delegateCall(executor, executorData);
```

OrderValidator.sol

```
// executor and executorData are not included in the signed payload
OperatorSignature(Order order, address user, uint256 nonce)
```

OKXExecutor.sol

```
IERC20(fromToken).forceApprove(dexTokenApprove, amount);
```

Remediation:

Cryptographically bind the execution method to the operator signature.

Extend the signed payload to include: $executor$ and $keccak256(executorData)$

```
OperatorSignature(
    Order order,
    address user,
    uint256 nonce,
    address executor,
    bytes32 executorDataHash
)
```

The issue was fixed in [65b53dd9198d867a9000b7b51baa90dbe7df3efd]

2.3 Medium

Not found

2.4 Low

Not found

CONCLUSION

The project demonstrates a solid and well-structured architecture with a clear separation of responsibilities between swap execution, fee collection, and referral reward accounting.

The primary security issue identified during the audit—related to user-controlled execution data not being cryptographically bound to operator authorization—was fixed by extending the operator signature to cover execution parameters. This change aligns on-chain behavior with backend assumptions and strengthens the protocol's security.

After applying the recommended fixes, no high-risk issues remain.

REFERENCES